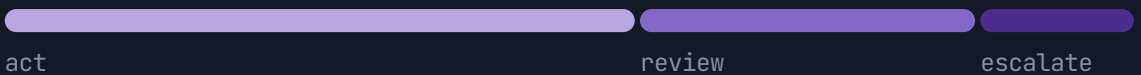


Decide, Don't Generate



Jev, System One Models, and the
Decision Layer of Agentic AI

Runnable Labs • 130+ figures • no API key needed

Sridhar Mukkandi

Table of contents

Preface	vii
How to read this book	ix
Prologue: the night shift	xii
I Prerequisites: probability and decisions	xv
1 What “learning” means	1
2 Probability, and how a machine learns it	13
3 Calibration: when 0.8 really means 80%	23
4 From probabilities to actions	34
II Prerequisites: networks, language models and agents	43
5 Deep learning in one chapter	45
6 How an LLM writes, and structured outputs	54
7 RAG, agents, and where they break	63
III Jev in depth	73
8 System 1 and System 2	75
9 Inside Jev: what we know and what we don’t	83
10 The type system: choice, score, noul	92
11 Testing Jev’s calibration yourself	101
12 The Jevons paradox of decisions	110
IV The decision layer	121
13 The bake-off: six ways to make a decision	123

14 Act, review, or escalate	133
15 A catalogue of decision patterns	147
 V Building	 159
16 First calls, and the mock that makes them free	161
17 A hybrid agent: Jev decides, the LLM reasons	170
18 Case study: SOC alert triage	178
19 An applications gallery	186
20 Build your own System One model	195
21 Capstone: a production decision service	204
 VI What changes now	 213
22 What changes now	215
 Appendices	 222
The Python you'll see	222
Glossary	226
Cheat sheets	230
Bibliography	233
Index	237
About the Author	239

Calibration: when 0.8 really means 80%

When you can measure what you are speaking about, and express it in numbers, you know something about it.

— Lord Kelvin, lecture on electrical units of measurement, 1883

Here's a question that sounds silly until it costs you money:

“If a model says 80%, does that mean 80%?”

Your first answer is probably “of course”: it's a probability, and that's what the number is for.

But a number is just a number. A model can say 0.8 about alerts that turn out to be attacks half the time, or nineteen times out of twenty. Nothing in the maths of the model forces its 0.8 to match reality. It matches only if we built it well, trained it on the right data and checked it. Even then, it can slowly go wrong over time.

This chapter is about checking, the most important habit in the book. Every threshold in Chapter 4, every policy in Chapter 14 and every claim about Jev in Part III depends on it.

BY THE END OF THIS CHAPTER

- Say what it means for a model to be *calibrated*, and why that's different from being accurate.
- Draw and read a reliability diagram, and compute the expected calibration error (ECE).
- Recognise the everyday mistakes that break calibration, including rebalancing your training data.
- Fix a miscalibrated model with Platt scaling, temperature scaling or isotonic regression, and know which to use.
- Check calibration within the groups you'll actually act on.

The weather forecaster's secret

Let's go back to the weather.

People love to complain about weather forecasters. But in the 1970s, researchers checked American forecasters' chance-of-rain forecasts against what actually happened. They found something surprising. When the forecasters said "30% chance of rain", it rained on about 30% of those days. When they said 70%, it rained about 70% of the time. Across the whole range, what they said and what happened matched almost perfectly (Murphy and Winkler 1977).

Compare that with most people. Ask someone for a range they're "90% sure" contains the answer to a quiz question. The true answer falls inside it far less often than 90% of the time. As a rule, people are too sure of themselves.

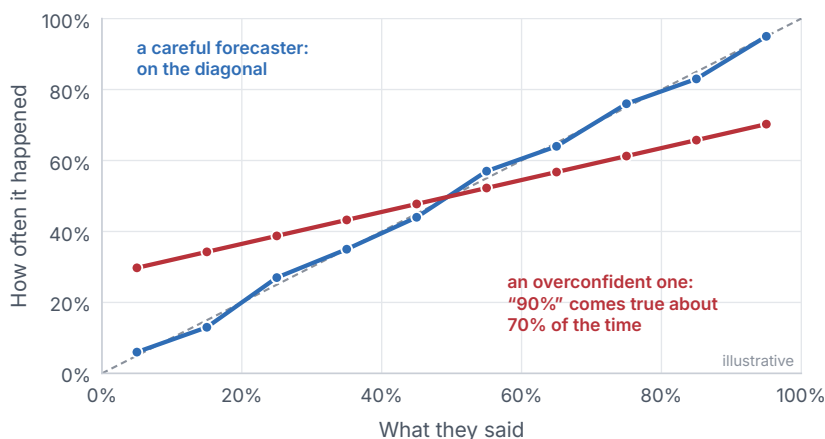


Figure 3.1. Two forecasters. For each thing they said ("30%", "70%" and so on), we count how often it actually happened. The careful forecaster sits on the diagonal: their 70% happens about 70% of the time. The overconfident one says 90% about things that happen only about 70% of the time.

The weather forecasters aren't good because they're always right. They can't be. They're good because their numbers mean what they say. That property has a name: **calibration**.

A model is calibrated when its 80% happens about 80% of the time.

And, importantly, you can *check* calibration with nothing more than grouping and counting. Collect everything the model said "about 80%" to, and count how often it happened. A calibrated model's probabilities are honest: they mean what they say.

Good at ranking, badly calibrated

Before we check anything, you need one idea, because people confuse it all the time.

There are two separate things you might want from a model's probabilities.

The first is **ranking**: does the model give attacks higher numbers than harmless alerts? If you sort alerts by the model's score, do the real attacks rise to the top? The **AUC** measures this. It's the chance that a real attack, picked at random, scores higher than a harmless alert picked at random. An AUC of 1 means every attack scored above every harmless alert. An AUC of 0.5 means the scores are no better than tossing a coin.

The second is **calibration**: when the model says 0.3, do three in ten of those alerts turn out to be attacks?

The two are separate. A model can rank perfectly and still be wrong about how sure it is.

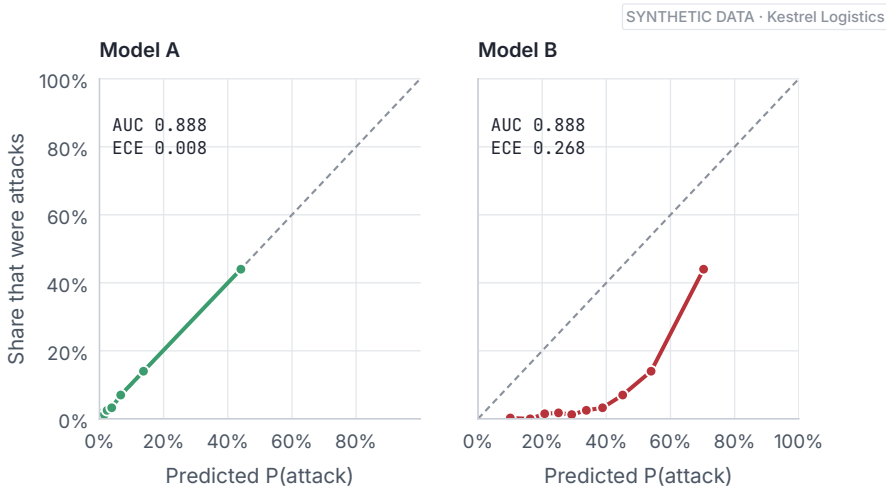


Figure 3.2. Two models, identical at ranking (same AUC) but not at calibration. Model A is the logistic regression from Chapter 2. Model B is the same model with its probabilities distorted: it still sorts the alerts in the same order, but its numbers no longer mean anything.

Figure 3.2 shows two models with the same AUC, 0.888. Model B's numbers are badly wrong: it says 40% about alerts that are attacks only a few percent of the time. If you only ever sort alerts, Model B is as good as Model A. But as soon as you put a threshold on its numbers, use them in a cost, or show them to an analyst, it'll mislead you.

This is the trap: most leaderboards and model comparisons report ranking measures like AUC, accuracy or F1. (F1 is one score that balances two

things: how many of the cases a model flags are real, and how many of the real cases it flags.) Calibration is often not measured at all. But for a decision system, it's usually what matters most.

The reliability diagram

So how do we check? The same way the weather researchers did.

1. Take a set of predictions where you know what happened: our test alerts from Chapter 2.
2. Sort them into bins by what the model said: 0–10%, 10–20% and so on.
3. In each bin, compare two numbers: the model's *average* prediction and the share of alerts that really were attacks.
4. Plot one against the other.

That plot is called a **reliability diagram**, and you'll see a lot of them from here on.

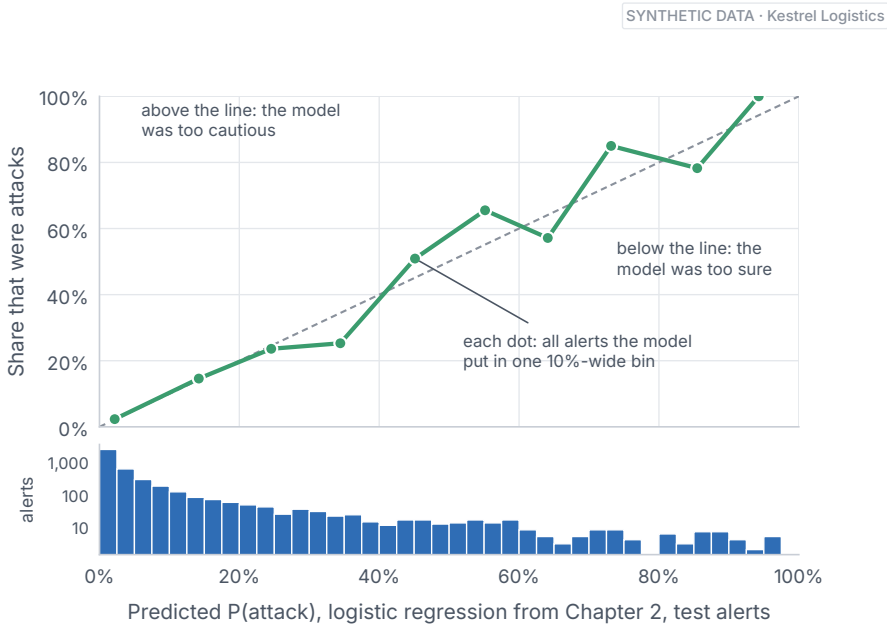


Figure 3.3. The reliability diagram of our Chapter 2 model on alerts it never saw. Points on the dashed diagonal mean the model's numbers can be trusted as they are. Underneath, a histogram (log scale) shows how many alerts fall in each part of the range: most are near zero, and only a few are above 50%.

Look at Figure 3.3 carefully, top and bottom together, because people forget the bottom panel. Our model's points are near the diagonal where there's lots of data. At the right end, the dots jump around, and the histogram tells you why: only a few dozen alerts are there. A dot based on

twelve alerts is weak evidence, as Chapter 2 warned. Always look at the counts before you worry, or before you relax.

TRY IT

Build the diagram's table yourself: bin, average prediction, observed rate, count.

```
alerts = soc.load()
train, calib, test = soc.split(alerts)
F = lambda d: soc.feature_matrix(d).to_numpy()
model = LogisticRegression(C=1e4, max_iter=5000).fit(
    F(train), train.malicious)
p = model.predict_proba(F(test))[:, 1]
y = test.malicious.to_numpy()

bins = np.clip((p * 10).astype(int), 0, 9)
table = (pd.DataFrame({"bin": bins, "said": p, "happened": y})
        .groupby("bin")
        .agg(said=("said", "mean"), happened=("happened", "mean"),
            n=("said", "size")))
print(table.round(3).head(6))
```

	said	happened	n
bin			
0	0.022	0.023	3266
1	0.142	0.146	315
2	0.246	0.236	144
3	0.344	0.253	87
4	0.451	0.509	55
5	0.551	0.655	58

One number: ECE

Pictures are the best way to *see* calibration. But sometimes you need a single number, to compare models or to raise an alarm when calibration gets worse. The most common is the **expected calibration error**, or ECE. It's the average gap between “said” and “happened” across the bins, weighted by how many predictions each bin holds (Naeini et al. 2015).

```
print(f"ECE: {cal.ece(p, y):.4f}")
```

```
ECE: 0.0077
```

Our logistic regression's ECE is 0.008. That means its probabilities are off by less than one percentage point on average. Good. Model B's is 0.27.

ECE has weaknesses worth knowing. It depends on how you choose the bins. And because it's an average, it can hide very bad calibration in a small region. The high end, where the dangerous alerts are, can be badly calibrated while the rest of the data is fine. Report ECE, but always look at the picture too.

GOING DEEPER optional

The **Brier score** is the average squared gap between a predicted probability and what happened (1 or 0). Allan Murphy showed that it splits into three parts (Murphy 1973): $\text{Brier} = \text{reliability} - \text{resolution} + \text{uncertainty}$. **Uncertainty** is how hard the problem is to begin with (it depends only on the base rate). **Resolution** is how well the model's predictions separate cases that turn out differently; it's close to ranking skill. **Reliability** is the calibration error: the squared gap between said and happened, bin by bin. For our model: reliability 0.0005, resolution 0.0212, uncertainty 0.0698. A model can improve its Brier score by ranking better or by being better calibrated. The three parts tell you which.

How calibration breaks

If log loss rewards calibrated probabilities, as Chapter 2 said, why would a model ever come out miscalibrated? Because the way it's trained can mislead it. Here are the usual causes.

Rebalancing the data. This is one of the most common causes, and people usually do it with good intentions. Attacks are rare, only 8% of alerts. So someone decides the model needs to “see more attacks” and trains it on a 50/50 mix: every attack, plus the same number of harmless alerts chosen at random. The ranking is still fine. But the model has learned from a world where attacks are *common*, and it correctly reports probabilities for that world, not ours.

The rebalanced model's average prediction is 29%. The real attack rate is 8%. Put a threshold on those numbers and you'll send your analysts far more alerts than they can handle.

Overfitting. Flexible models that memorise, like the deep decision tree from Chapter 2, push their probabilities towards 0 and 1. On the training data, they really were that sure. They come out overconfident on new data.

Scores that were never probabilities. Some models output a “score” that happens to be between 0 and 1 but was never trained with a proper scoring rule. Treat it as a ranking until you've checked it.



Figure 3.4. A model trained on 50/50 rebalanced data (red) thinks attacks are about ten times more common than they are, and its probabilities sit far below the diagonal. After Platt scaling fitted on a separate calibration set (green), the same model is well calibrated again.

Big modern networks. Surprisingly, very large neural networks trained to sort things into classes tend to be *overconfident* (Guo et al. 2017). Chapter 5 looks at why. The “confidence” an LLM states in words is a different thing again; Chapter 6 measures it.

The world changing. A model calibrated on last month’s base rate is miscalibrated on the day an attack campaign doubles the number of attacks. Chapter 14 shows this happening.

Fixing it

The good news: a model that ranks well but is badly calibrated is usually easy to fix. You don’t retrain it. You add a small correction on top.

The method is always the same. Keep back a **calibration set**: data the model never trained on. Run the model on it, and learn a simple function that turns the model’s raw probabilities into calibrated ones. Then check the result on the *test* set, which neither the model nor the correction has seen.

There are three common corrections, and they differ in how flexible they are.

Platt scaling fits a tiny logistic regression on top of the model’s log-odds. It learns two numbers: a stretch and a shift (Platt 1999). It’s the usual first choice.

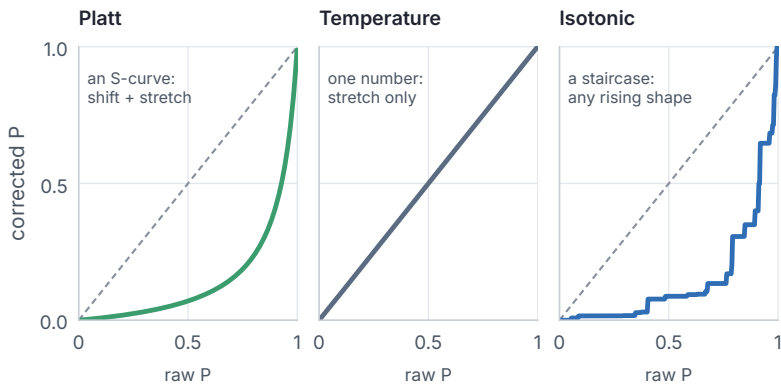


Figure 3.5. The three corrections, each fitted to the rebalanced model on the calibration set. Platt scaling learns an S-curve that can shift and stretch. Temperature scaling learns one number that only stretches around the middle, and here it barely changes anything. Isotonic regression learns any rising staircase.

Temperature scaling fits just one number, T , and divides the log-odds by it (Guo et al. 2017). A T above 1 softens overconfident predictions; a T below 1 sharpens predictions that are too cautious. It's the standard fix for overconfident neural networks. It's also what people usually mean by calibrating an LLM classifier.

Isotonic regression fits a staircase that only ever goes up (Zadrozny and Elkan 2002). It can fix almost any shape, but it needs more data, and it can overfit a small calibration set.

```
pos = train[train.malicious == 1]
neg = train[train.malicious == 0].sample(len(pos), random_state=0)
bal = pd.concat([pos, neg])
model_b = LogisticRegression(C=1e4, max_iter=5000).fit(
    F(bal), bal.malicious)
pb = model_b.predict_proba(F(test))[:, 1]
pb_cal = model_b.predict_proba(F(calib))[:, 1]

print(f"{'no fix':>12}: ECE {cal.ece(pb, y):.3f}")
for name, fixer in [("Platt", cal.Platt()),
                    ("Temperature", cal.Temperature()),
                    ("Isotonic", cal.Isotonic())]:
    fixer.fit(pb_cal, calib.malicious)
    print(f"{name:>12}: ECE {cal.ece(fixer(pb), y):.3f}")
```

```
no fix: ECE 0.216
Platt: ECE 0.010
Temperature: ECE 0.215
Isotonic: ECE 0.013
```

Platt and isotonic fix the rebalanced model almost completely. Temperature scaling doesn't help at all, and the reason teaches you something. Rebalancing didn't make the model overconfident. It *moved* every prediction up, as if attacks were common. A temperature can only stretch or squeeze predictions around 0.5. It can't move them all down. Platt can, because it has a shift as well as a stretch.

Temperature fixes overconfidence. It can't fix a wrong base rate.

Calibrated on average, wrong for one group

There's one more trap, and it's the one that causes problems in real systems.

A model can be well calibrated *overall* and badly calibrated for one group. Take a version of our model trained without knowing which detection rule fired. It can't tell an email alert from an alert on a laptop or server (an "endpoint").

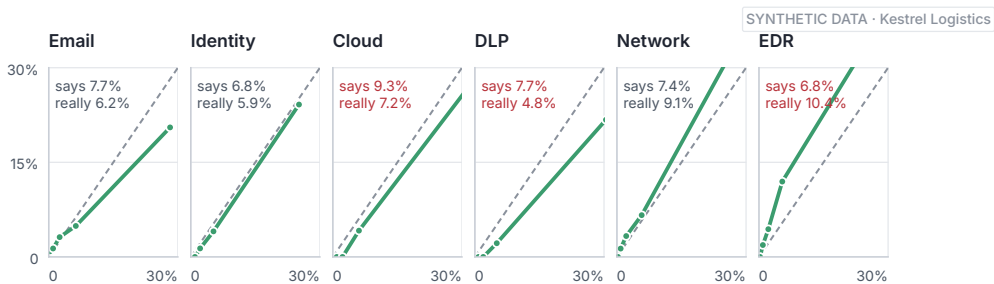


Figure 3.6. Calibration checked separately for each alert source. Across all sources together, this model is well calibrated, with an ECE of about half a percentage point. But it overestimates the risk of DLP and Cloud alerts and underestimates EDR alerts, which really are more dangerous here. Averaging hides both mistakes.

Its overall ECE is excellent: 0.005. Endpoint alerts come from EDR (endpoint detection and response) tools. For them, it says 6.8% when the truth is 10.4%. DLP (data loss prevention) alerts warn that company data may be leaving the building. For them, it says 7.7% when the truth is 4.8%. The estimates that are too high and too low cancel out in the average.

If your policy treats every source the same way, you'll send too many harmless DLP alerts to the queue and close too many real EDR ones. The rule is simple: **check calibration inside every group you'll treat differently**, and inside every group where a mistake would be especially costly. That means sources, customer tiers, languages and regions. It also means groups of people, such as age or gender, when a model affects people's lives.

✚ SET THE THRESHOLD

Kestrel auto-closes any alert under 3%. The model above says EDR alerts average 6.8% risk, but they're really 10.4%. About how much more often than planned will a real EDR threat be auto-closed? Name two ways to fix this: one that changes the model, and one that changes only the policy.

(About 50% more often. You could add the source as a clue, or calibrate separately for each source, so the model is calibrated for every source. Or you could keep the model and set a stricter act threshold for EDR alerts. The first is better; the second is quicker.)

⚠ WHERE THIS BREAKS

Calibration is always measured *on some data*, and it only holds for data like that. A model calibrated on last quarter's alerts can become less calibrated as tools, attackers and staff change. Isotonic regression fitted on a few hundred examples will overfit and produce rough, overconfident corrections. And calibration says nothing about any *single* prediction: a perfectly calibrated 70% is still wrong three times in ten. Keep a fresh calibration set, refit on a schedule, and keep watching the diagram.

“

The first principle is that you must not fool yourself, and you are the easiest person to fool.

— Richard Feynman, Cargo Cult Science, 1974

Where we are

The weather forecasters showed us that calibration can be checked. Then two models with the same ranking and very different calibration showed us why checking matters. Grouping and counting gave us the reliability diagram, and a well-meant rebalancing broke calibration. A small correction fitted on held-out data fixed it. And just when the average looked fine, the EDR alerts reminded us to look inside every group we'll treat differently.

When Part III tests Jev's own calibration, it'll use these same tools on the mock, and measure it against everything in this chapter.

EXERCISES

1. Explain to a colleague, without technical words, the difference between a model that ranks well and a model that's calibrated. Use a weather example.
 2. In the lab, train the rebalanced model with a 30/70 mix instead of 50/50. Is it more or less miscalibrated? Predict the answer before you run it.
 3. Why must the calibration correction be fitted on a separate calibration set, not on the training set? What would go wrong?
 4. Fit isotonic regression using only 200 alerts from the calibration set. Plot its staircase. What do you notice, and what does that tell you about when to prefer Platt?
 5. Name three groups in a system you know where calibration might differ, and one decision in each that would go wrong if it did.
-

Next: we finally have calibrated probabilities. Chapter 4 turns them into actions: what each mistake costs, where the thresholds go, and how to see the whole trade-off on one page.

The bake-off: six ways to make a decision

There are two ways of constructing a software design: One way is to make it so simple that there are obviously no deficiencies, and the other way is to make it so complicated that there are no obvious deficiencies.

— C. A. R. Hoare, Turing Award lecture, 1980

Every SOC has the same argument, and it never really ends.

The senior analyst says the rules are fine. People who know the attacks wrote them, and you can read every one. The data scientist says a trained model would beat the rules, if only someone would label enough alerts. The new engineer has connected an LLM to the ticket system and says it reads alerts better than either of them. And now someone has read about Jev.

Each of them is right about something. The argument never ends because they're judging different things. One cares about explaining decisions, one about accuracy, one about flexibility, and one about price. Nobody puts all the numbers on one page.

So let's do that: six methods, one dataset, one fair contest, and a scorecard with every column that matters. We'll call it a "bake-off", the name for a contest where everyone makes the same thing. I'll tell you now that Jev doesn't win every round.

BY THE END OF THIS CHAPTER

- Compare rules, logistic regression, a trained text classifier, LLM structured output, LLM-as-judge and Jev on the same decision.
- Score them on ranking, calibration, labels needed, speed, price and stability, not on one number.
- See where Jev loses, and why that's useful to know.
- Tell which results come from how the synthetic contestants were built, and which would also appear in your own test.
- Pick a method, or a combination, from a scorecard.

The contestants

The decision is Kestrel’s: is this alert a real threat? Every method gets the three history weeks to train or tune on: 14,973 alerts with analysts’ verdicts. Then every method is scored on the fourth week, the “live week”: 5,027 alerts it has never seen, of which 412 are real.

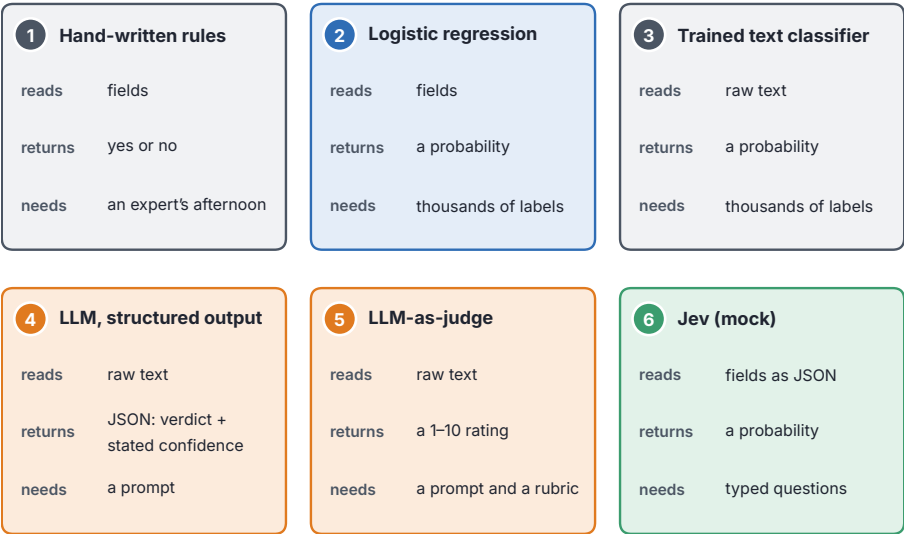


Figure 13.1. The six contestants: what each one reads, what it returns, and what it needs before it can start.

Each one reads what it would normally read in practice (Figure 13.1). The rules and the logistic regression get the alert’s structured fields from the SIEM. A SIEM (security information and event management system) is the tool that collects the company’s logs and raises the alerts. The text classifier and both LLM methods get the raw alert text. Jev gets the fields as a JSON state, the same information the logistic regression sees.

The **text classifier** needs a short explanation. It stands in for a fine-tuned model. It’s a classic classifier that counts words (word weights plus logistic regression), trained on the history labels. It re-trains in seconds on any laptop. A fine-tuned transformer, even a small one, would usually read the text better. So treat this row as the lowest result a trained text model should get.

One warning before any results, and it’s the most important paragraph in this chapter. Both LLM contestants are the book’s mock, and I built it to read alerts *less accurately* than mock Jev (the repository’s docs/mock-design.md explains how). So when the LLM ranks alerts worse than Jev below, that’s because of how the mocks were made. It isn’t

evidence about real LLMs or real Jev. What you *can* rely on is the pattern of the other results. That means how stated confidences behave, how often free-text JSON breaks, what happens when you ask twice, and what labels cost. Those come from how each kind of method works, not from choices I made in the mocks.

Round one: ranking

The first question is the one people usually ask: which method puts the real threats at the top?

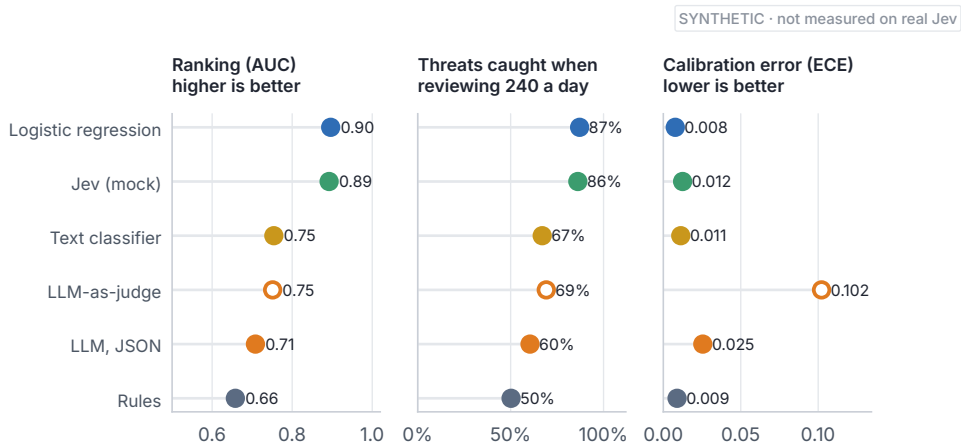


Figure 13.2. Three views of how well each method separates threats from harmless alerts, on the live week. Left: ranking quality (AUC). Middle: the share of all real threats found if the team reviews the top 240 alerts each day. Right: calibration error, where lower is better.

Logistic regression wins round one, by a small margin: an AUC of 0.896 against Jev’s 0.892 (Figure 13.2). With Kestrel’s review capacity of 240 alerts a day, it catches 87% of real threats; Jev catches 86%.

That isn’t luck. The logistic regression was trained on fifteen thousand of Kestrel’s own labelled alerts, on clean structured fields. And the decision depends on those fields in a nearly straight-line way. This is exactly the kind of problem classic machine learning is good at. A general model reading the same fields, with no training on Kestrel at all, shouldn’t be expected to beat it here, and it doesn’t.

The rules come last on ranking, with an AUC of 0.66. The reason is simple: a rule only says yes or no. Among the hundreds of alerts it says yes to, it has no way to say which to look at first.

Give Jev the raw text instead of the fields, and its AUC drops to 0.856. That’s still well ahead of the trained text classifier’s 0.754. A classifier that counts words is worst at reading “threat intel score: 0.82” as a number.

Round two: calibration

Ranking is only half the job. Chapter 14 will put thresholds on probabilities, and thresholds only work if the probabilities mean what they say.

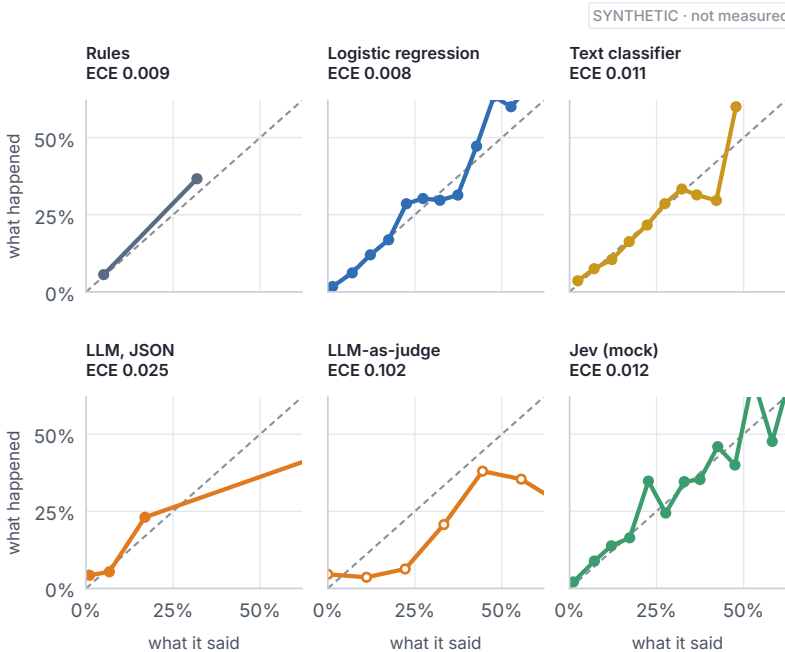


Figure 13.3. Reliability diagrams for each method on the live week. On the dashed line, what a method said matches what happened. Most methods here are close. The LLM's stated confidences are grouped on a few values; the judge's ratings, read as probabilities, are far too high in the middle.

Most of the contestants are close to the diagonal (Figure 13.3). The logistic regression was fitted with log loss on this company's data. Chapter 2 said that method produces calibrated probabilities. Mock Jev was designed to be well calibrated at Kestrel. Chapter 11 showed that you should check this at your own company, not assume it.

The two LLM methods are the interesting ones.

The LLM's stated confidence came from the JSON it wrote: "confidence": 0.95. Across five thousand alerts, it used only 6 different values, mostly 0.9, 0.95 and 0.99. Chapter 6 predicted this. A number that a model *writes* is text, and text tends to use round numbers. So it can't rank alerts finely. That's why the LLM's AUC is the lowest of the methods that give probabilities.

In the LLM-as-judge method, the LLM rates each alert from 1 to 10. That rating was never meant to be a probability. Reading it as one gives the worst calibration in the contest: 0.102. You could fix that with Platt scaling

on a few hundred labels, as Chapter 3 showed. But then it's no longer a method that needs zero labels.

Round three: how many labels?

“Logistic regression wins” comes at a cost: fifteen thousand labels. What if you have fewer?

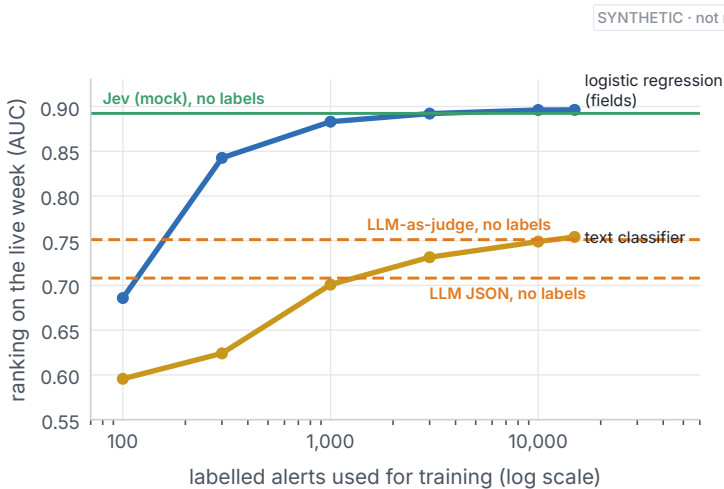


Figure 13.4. Ranking on the live week as the trained methods see more labelled alerts, averaged over random samples. Jev and the LLMs need no labels to start, so they're flat lines. Logistic regression catches up with Jev at about 3,000 labels.

Figure 13.4 is the chart I'd show the data scientist in that argument. Logistic regression needs about 3,000 labelled alerts to catch up with Jev. With a few hundred, it's clearly behind.

Where would 3,000 labels come from? Not from analysts' reviews alone: at 240 a day, that's nearly two weeks of verdicts. Kestrel's labels are *what finally happened*: the incident that was confirmed or ruled out, the user who said “that was me”, the alert nobody followed up that turned out to be nothing. Every alert gets a label sooner or later. So at about 700 alerts a day, 3,000 labels is about four days of history, and Kestrel has that. A team that's just starting doesn't, and neither does a new decision nobody has labelled yet.

The text classifier never catches up, even with every label we have. Counting words doesn't capture the numbers in the text.

Labels are the hidden cost of trained models. A method that needs none can start today. A method that needs thousands has to wait for them, and needs them again when things change.

Round four: speed and price

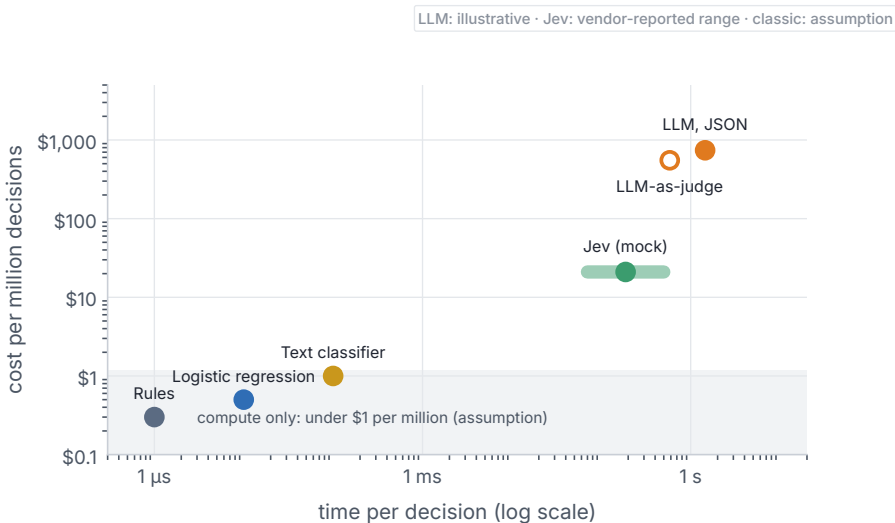


Figure 13.5. Time and cost per decision for each method, on log scales. Classic models run in millionths of a second, for fractions of a cent per million decisions (an assumption: computing cost only). The LLM figures are this book’s illustrative assumptions; Jev’s are vendor-reported.

Nothing beats a rule on speed or price, and the classic models are close behind (Figure 13.5). They run in millionths of a second, on computers you already own. Jev’s vendor-reported figures put it in between: about \$21 per million decisions, in tens to hundreds of milliseconds. The illustrative LLM costs about \$740 per million and takes about 1.4 seconds. That’s a little more than Chapter 9’s figures. The bake-off’s JSON answer has four fields (verdict, confidence, rule and threat-intel score), so it’s about 60 output tokens rather than 40.

If a rule or a logistic regression can make a decision well, use one of them. It’s the cheapest and fastest option by a factor of a thousand or more. Jev’s price advantage is against *LLMs*, not against everything.

Round five: the same answer twice

Ask the LLM the same question five times at a temperature of 0.7, and 5.2% of verdicts change at least once (Figure 13.6). At temperature 0, the mock

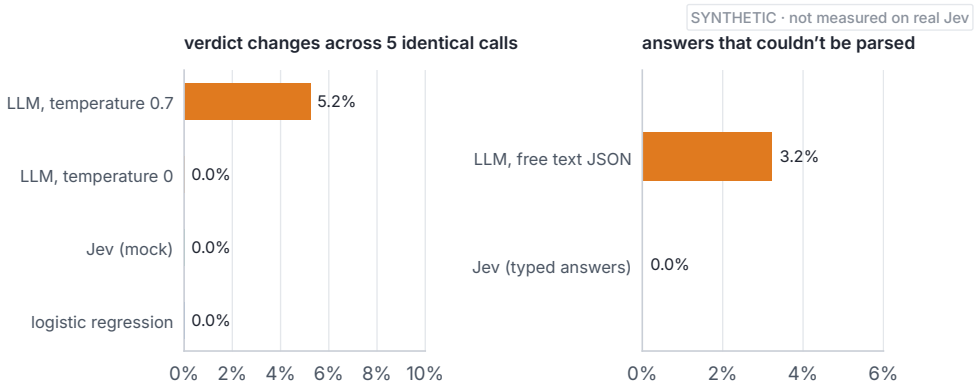


Figure 13.6. Left: the share of alerts whose verdict changed across five identical calls. Right: the share of answers that couldn't be parsed at all.

always gives the same answer. Real APIs mostly do too, though not always. And 3.2% of its JSON answers were so broken that they couldn't be parsed. We used the base rate for those, but in a real system each one is an error your code has to handle.

Jev's typed answers can't fail to parse: the probabilities arrive as numbers in a fixed shape. The mock always gives the same answer to the same question. You should check whether real Jev does.

The scorecard

Now put everything on one page.

SYNTHETIC · not measured on real Jev

	ranking (AUC)	caught at 240/day	calibration (ECE)	cost per million	time per decision	labels needed	same answer twice?	explains itself?
Rules	0.66	50%	0.009	< \$1	1 μs	none	yes	fully
Logistic regression	0.90	87%	0.008	< \$1	10 μs	~3,000	yes	weights
Text classifier	0.75	67%	0.011	\$1	100 μs	15,000+	yes	word weights
LLM, JSON	0.71	60%	0.025	\$740	1.4 s	none	not always	in words*
LLM-as-judge	0.75	69%	0.102	\$552	583 ms	none	not always	in words*
Jev (mock)	0.89	86%	0.012	\$21†	70–500 ms*	none‡	yes (mock)	no

Bold: best in column. † vendor-reported. ‡ none to start; about 300 to check and calibrate. Classic costs: compute only (assumption).
* reasons written after the fact, which may not be how the verdict was reached (Chapter 6). LLM figures: illustrative.

Figure 13.7. The whole contest on one page. Bold marks the best in each numeric column. Every accuracy number is synthetic; every cost and time is an assumption, an illustration, or vendor-reported, as marked.

Read Figure 13.7 by column, not by row, and ask which columns *your* decision cares about.

If you have thousands of labels, clean fields and a decision that doesn’t change much, logistic regression is hard to beat. It ranks best, it’s calibrated, it’s nearly free, and you can read its weights. Jev clearly loses there.

If you need to explain every decision to an auditor, rules clearly win the last column, and nothing else comes close.

If you have no labels, raw text, or many different decisions to make, Jev’s row is the strongest. It starts with zero labels and ranks nearly as well as the trained model. After a calibration check, you can put thresholds on its probabilities. Its answers come in a shape that can’t break, and it costs a small fraction of an LLM call.

And the LLM? In this contest it’s the weakest at deciding, partly because of how the mock was built. But notice what it does that no other row even tries: it reads anything, and it *writes*. That’s why Chapter 15’s first pattern gives it the job of extracting and explaining, and gives the decision to something else.

 TRY IT

```

from jevkit import bakeoff as B    # the six contestants, one live week
r = B.run()
for m in B.METHODS:
    s = B.scores(r["probs"][m], r["y"])
    print(f"{B.NAMES[m]:>26}: AUC {s['auc']:.3f}, "
          f"ECE {s['ece']:.3f}")

```

```

Hand-written rules: AUC 0.658, ECE 0.009
Logistic regression: AUC 0.896, ECE 0.008
Trained text classifier: AUC 0.754, ECE 0.011
LLM, structured output: AUC 0.708, ECE 0.025
LLM-as-judge: AUC 0.751, ECE 0.102
Jev (mock): AUC 0.892, ECE 0.012

```

 WHERE THIS BREAKS

This is one decision, at one synthetic company, with mocks standing in for the LLM and for Jev. The accuracy gap between the LLM and Jev was built into the mocks. It says nothing about the real models. The text classifier is a weak stand-in for a fine-tuned model. And a single live week has only about four hundred real threats, so differences in the second decimal place of AUC could be chance. The value of a bake-off is the *method*. Run the same contest on your own data, with the real models, and fill in your own scorecard.

 SET THE THRESHOLD

A new team at Kestrel is taking over cloud-storage alerts, a decision nobody has labelled. They'll get about 300 labels a week from their own reviews. Which method would you start with, and when would you think about switching? What would you use the first 300 labels for?

(Start with a method that needs no labels, Jev in this contest. Use the first 300 labels to check and fix its calibration, as in Chapter 11. After three or four weeks of labels, train a logistic regression next to it and compare them on the next week. If the two agree closely, keep the cheaper one. If they disagree, look at the alerts where they disagree.)



For any algorithm, any elevated performance over one class of problems is offset by performance over another class.

— David Wolpert and William Macready, No Free Lunch Theorems for Optimization, 1997

Wolpert and Macready proved this about search and optimisation (Wolpert and Macready 1997), but the idea fits any bake-off. There is no best method, only a best method for a particular decision.

Where we are

Six methods took the same test. The trained logistic regression won on ranking and calibration, because it had fifteen thousand labels on clean fields. It needed about three thousand of them just to catch up with a method that needed none. The rules won on explanation and price. The LLM's stated confidences turned out to be round numbers, and its free-text JSON broke often enough to matter. Jev's row was the strongest wherever labels were few or the input was raw text. None of that tells you what real models will do on your data. It tells you which columns to measure when you try.

EXERCISES

1. In the lab, write a better hand-written rule. How close to the logistic regression can you get? What did you have to add?
2. Fit Platt scaling to the LLM-as-judge ratings using 300 history alerts. What happens to its ECE? To its AUC? Why does one change and not the other?
3. Run the labels curve again with the text classifier using bigrams (pairs of words) only. Does it help? Why might a fine-tuned transformer do better?
4. Build a scorecard for a decision in your own work. Which three columns matter most? Which method would you try first?

Next: a probability isn't an action. Chapter 14 turns these numbers into three doors, act, review and escalate, and puts the thresholds where the costs say they belong.



That's the end of the sample.

The full book has 22 chapters, runnable labs for every one of them, and a free mock of Jev that needs no API key.

Code, labs and figures

<https://github.com/Mukkandi-Sridhar/JEVBook>

Contact

`sridhar.authorhub@gmail.com`